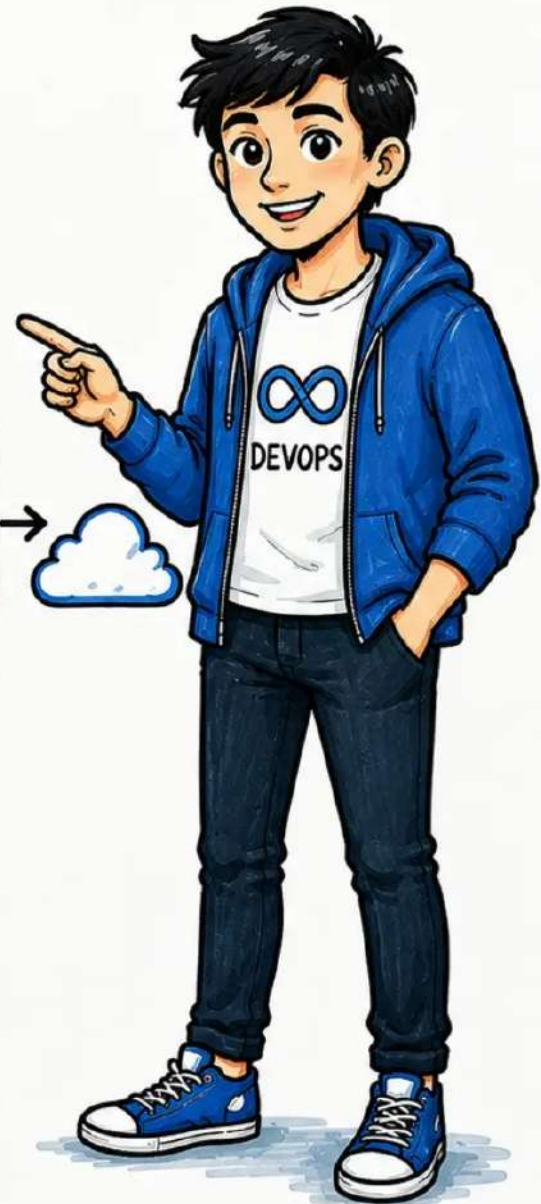
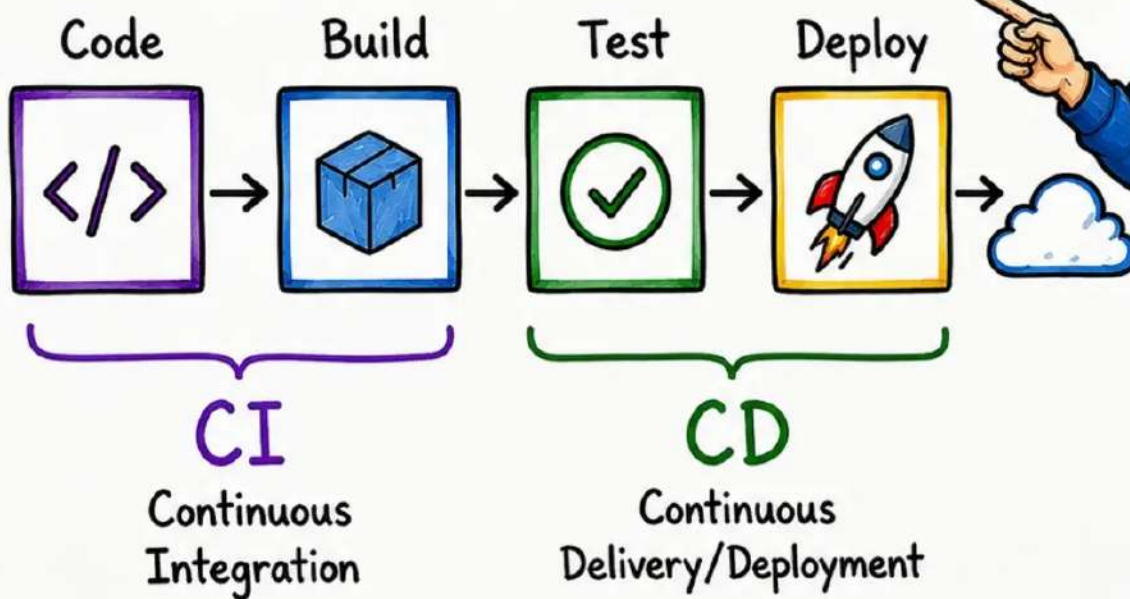


CI/CD

The automated path
from code to production.

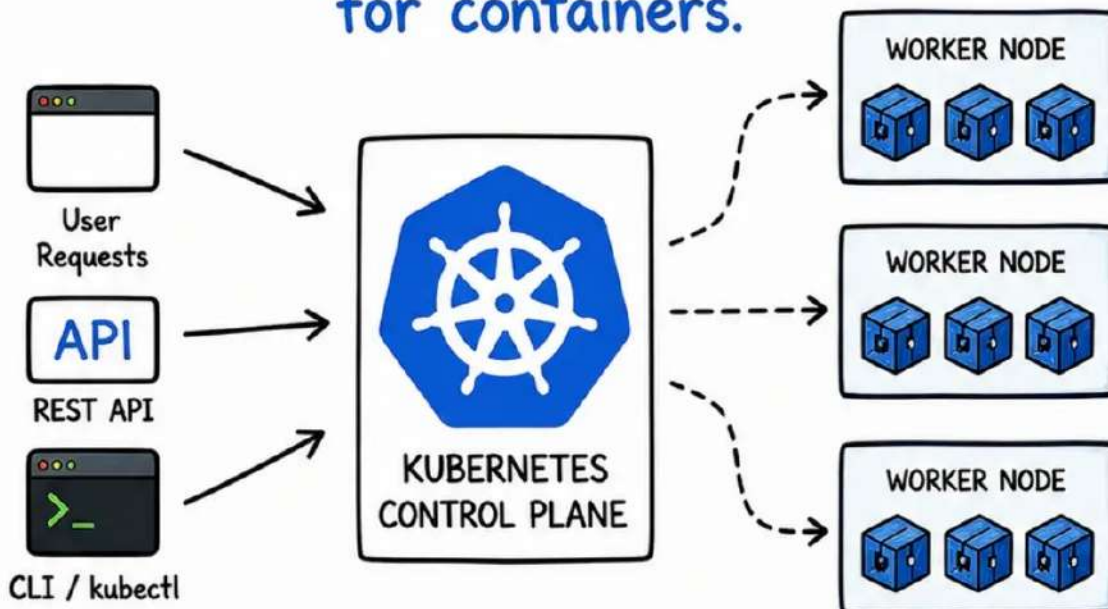


CI = Integrate code often

CD = Deliver changes safely and fast

Kubernetes

The traffic controller
for containers.



Scale

Automatically adjust based on demand.



Heal

Detect and fix issues automatically.



Deploy

Roll out changes quickly and safely.

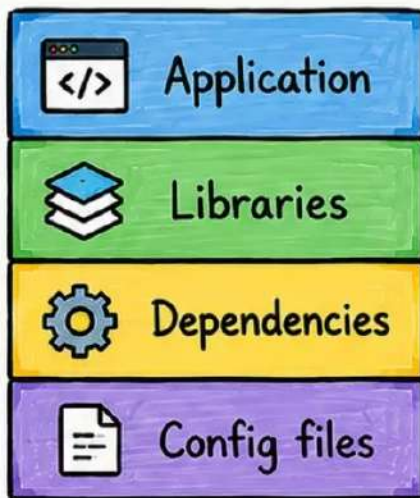


Kubernetes makes sure your containers are running **where they should**, and **everything stays healthy**.

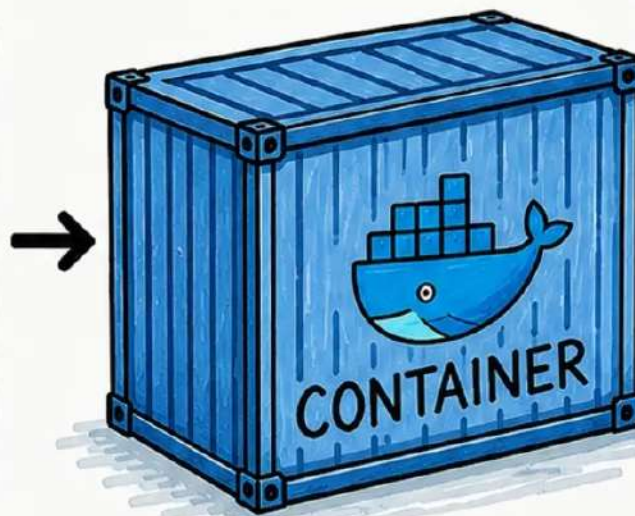
Container

A lightweight package that carries your app and everything it needs.

What your app needs



Packaged together



Containers are portable, consistent, and run the same **everywhere!**

Infrastructure as Code (IaC)

Build servers and infrastructure using code instead of clicking buttons.

❌ Manual Setup

VS.

✅ Infrastructure as Code

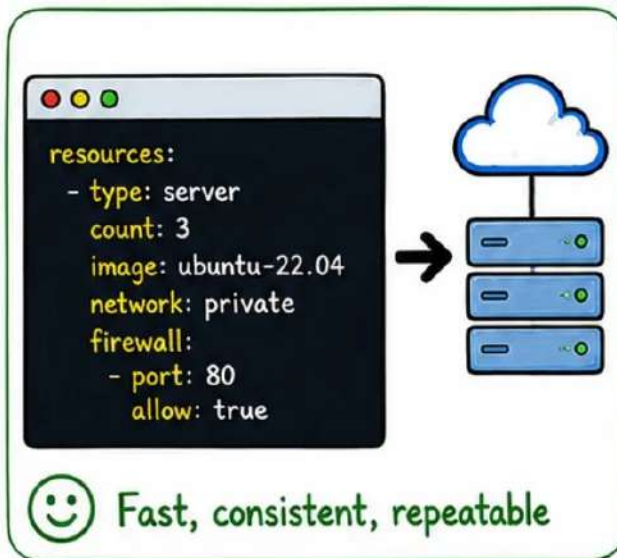


Click here...
Then here...
Oops!

CLOUD CONSOLE

- ✓ Create VM
- ✓ Configure Network
- ✓ Open Ports
- ✓ Install Software
- ... and more

☹️ Slow, error-prone, not repeatable

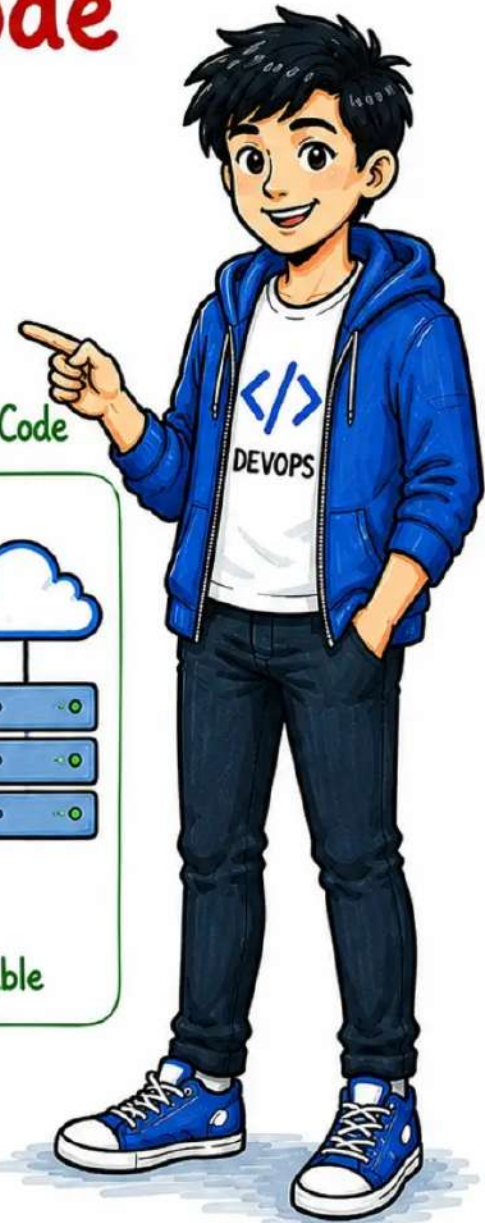


```
resources:  
- type: server  
  count: 3  
  image: ubuntu-22.04  
  network: private  
  firewall:  
    - port: 80  
      allow: true
```

☺️ Fast, consistent, repeatable

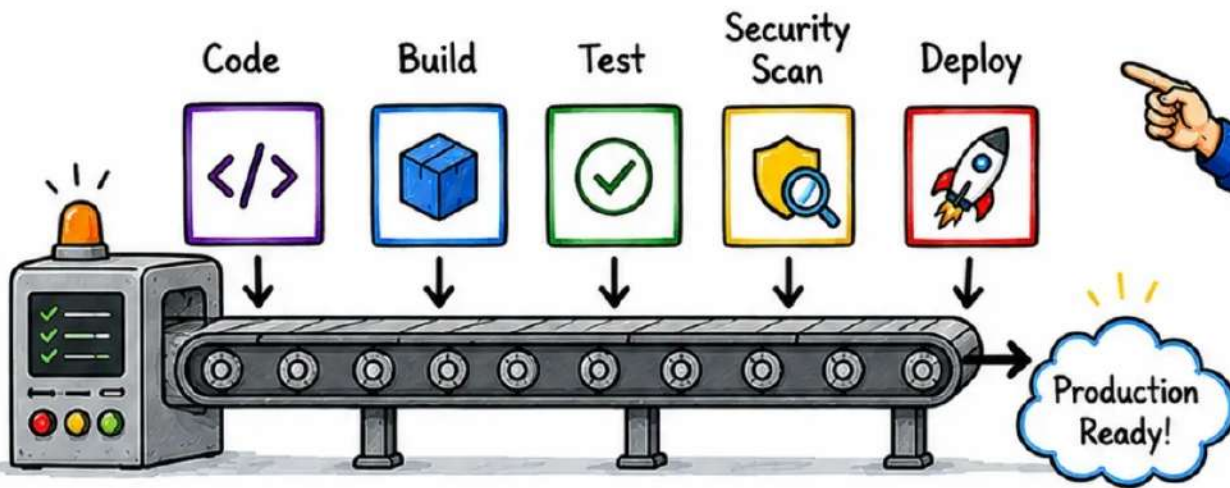


Write it once, use it everywhere.
Version it. Review it. Automate it.



Pipeline

A sequence of automated steps for your application.



Code

Write and commit code



Build

Compile and package the application



Test

Run automated tests to ensure quality



Security Scan

Find vulnerabilities before going to production



Deploy

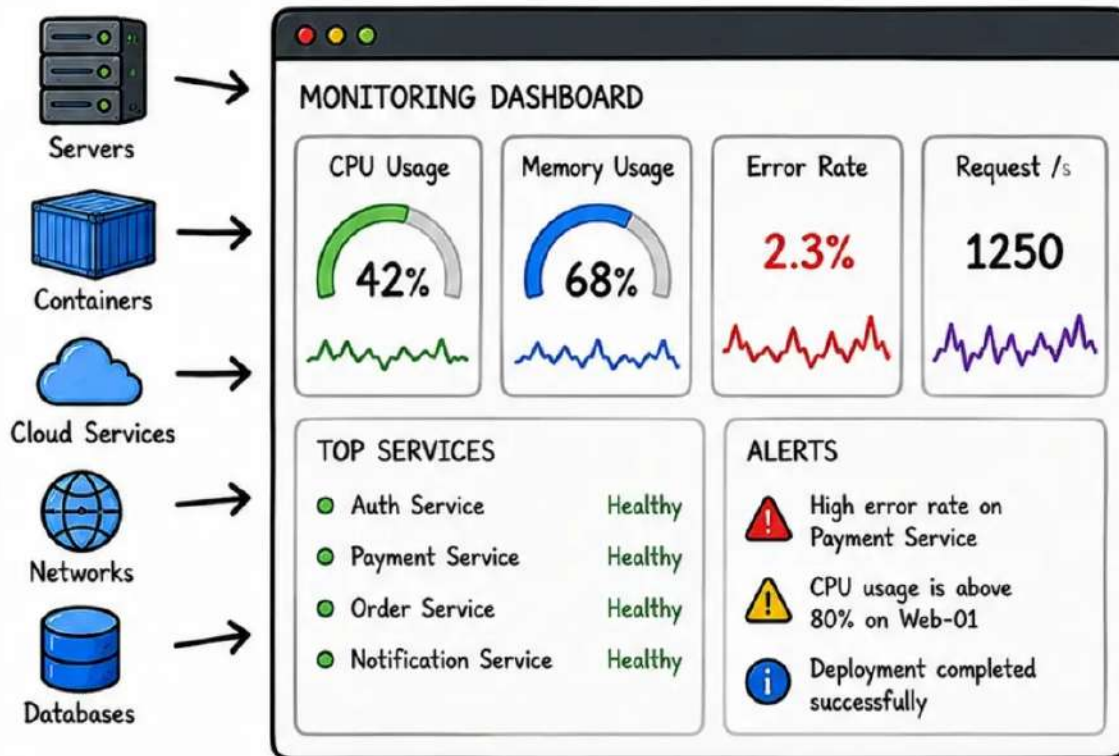
Release the application automatically



Pipelines help you deliver changes faster, safer, and more reliably.

Monitoring

Keeping an eye on your systems 24/7.



Monitoring helps you **detect issues early**, **minimize downtime**, and keep your applications **healthy**.



Issue Detected



Alert Triggered



Team Notified



Issue Resolved

Configuration Management

Keep your systems consistent and predictable.

The Problem

Manual changes lead to:

- ✗ Inconsistencies
- ✗ Drift
- ✗ Configuration Chaos
- ✗ Hard to scale

Server - 01



- ✓ Nginx
- ✓ Node.js
- ✓ PM2
- ✗ Firewall

Server - 02



- ✓ Nginx
- ✓ Node.js
- ✗ PM2
- ✓ Firewall

≠



The Solution

Use Configuration Management Tools



Ansible



Puppet



Chef



SaltStack

- ✓ Define desired state as code
- ✓ Automate configuration
- ✓ Ensure consistency everywhere

How It Works



1. Write desired configuration (as code)

2. Store in version control (Git)

3. Configuration management tool applies the changes

4. All servers are consistent and up-to-date

5. Predictable, reliable, and easy to manage



Configuration Management brings **order** to infrastructure chaos and makes **scaling** **safe and reliable**.

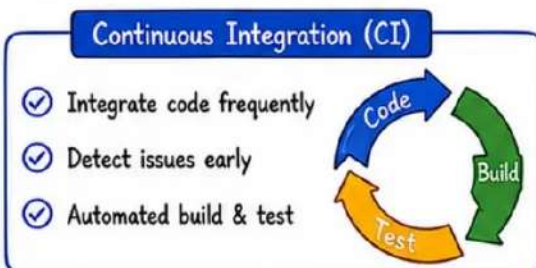
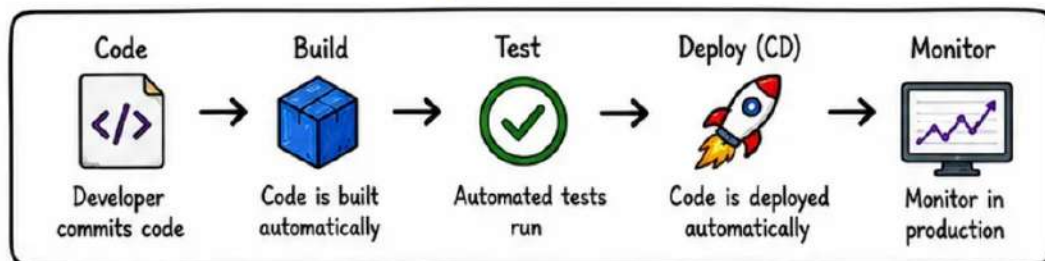
Remember:

If it's not in code, it doesn't exist.



CI/CD (Continuous Integration / Continuous Delivery)

Automate the build, test, and delivery of your code.



CI/CD turns manual, error-prone processes into automated, repeatable, and reliable workflows.

Build once.
Test always.
Deploy anywhere.

Infrastructure as Code (IaC)

Manage and provision infrastructure through code, not manual processes.

The Problem

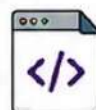
- ✗ Manual setup is slow and error-prone
- ✗ Hard to reproduce environments
- ✗ Inconsistent infrastructure
- ✗ Difficult to scale and maintain



The Solution

Use Infrastructure as Code

- ✓ Define infrastructure in code
- ✓ Version controlled
- ✓ Consistent and repeatable
- ✓ Automated provisioning
- ✓ Easy to scale and maintain



Infrastructure Code

How It Works



1. Write IaC (e.g., Terraform, CloudFormation)



2. Commit to Version Control (Git)



3. IaC Tool reads the code and plans



4. Review and approve the plan



5. Infrastructure provisioned automatically

Popular IaC Tools



Terraform



AWS CloudFormation



Pulumi



Ansible

Benefits



Consistency Everywhere



Faster Provisioning



Fewer Errors, More Reliability



Easier Scaling and Maintenance



Better Collaboration with Teams



IaC helps you treat infrastructure like software: **version it**, **test it**, and **deploy it** with confidence.

Remember:

If it's in code, it can be recreated.
If it's manual, it's fragile.



DevOps Culture

It's not a tool or a process.
It's a mindset and a way of working together.

What's the Problem?

- ✗ Silos between teams
- ✗ Blame culture
- ✗ Slow feedback loops
- ✗ Resistance to change
- ✗ Focus on tools, not outcomes



Silos slow everything down.

The DevOps Culture Mindset

- ✓ Collaboration (Work together, not in silos) 
- ✓ Ownership (Take end-to-end responsibility) 
- ✓ Automation First (Automate repeatable work) 
- ✓ Continuous Improvement (Inspect and improve always) 
- ✓ Customer Focus (Deliver value, not just features) 

The Outcome

-  Faster delivery
-  Higher quality
-  Happier teams
-  Lower risk
-  Better customer satisfaction



How a DevOps Culture Comes to Life



DevOps Culture is the glue that brings **people**, **processes**, and **technology** together to deliver amazing things!

Remember:

Tools and processes can be copied. Culture can't. Build a strong one!